

DPI343

Computer Organization and Assembly Language

Introduction

Lectures - prof. em. U. Sukovskis

Labs for local students - assoc. prof. P. Rusakovs

Labs for foreign students - assoc. prof. G. Alksnis

Autumn 2025



RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE

COAL

- **The goal of the study course** is to train and guide students in the organization of modern computer components so that at the end of the study course they would be competent to use the assembly language to work with them.
- **Tasks**
 - to learn basics of programming in the assembly language;
 - to strengthen the ability to read existing source code in the assembly language;
 - to master the processing of interruptions and the principles of operation of computer components;
 - strengthen skills in software development and debugging tools.
- **Learning outcomes. After completion of the course the student**
 - is able to independently implement simple programs in the assembly language and explain their operation
 - is able to analyse and explain the existing assembly source code and explain the operation of the operators used in it.
 - understands and is able to explain the principles of interrupt handling and operation of computer components (keyboard, video adapter, timer).
 - can use software development environment for program creation and debugging.

Tentative course outline

- Simple program, syntax basics. Programming tools
- Binary representation of numerical data
- CPU registers, memory addressing
- Arithmetic calculations
- Boolean operators. Conditions and branching. Loops
- String processing instructions
- Simple text input and output
- Procedures, parameter passing
- Interfacing with high level languages
- *Midterm Test 1*
- Interrupts
- Timer and sound
- Principles of low level video output
- Keyboard interrupt processing
- *Midterm Test 2*

This is not a complete assembly language programming course. The course covers basics of the language and provides basic skills. The course does not cover several assembler features such as floating point arithmetic, macro definition programming, etc.

The course has 3 Latvian credits (around 4.5 ECTS), which means that you have to spend a total of about 60 hours (about 3-4 hours per week) working independently on lecture materials (including examples of code) and with literature sources, and preparing for individual assignments. This amount of independent study working hours³ will be taken into account when evaluating results.

Previous knowledge that needs to be refreshed

- Number systems (bin, hex, dec),
- Representation of integers in binary and hexadecimal systems,
- Basics of computer architecture
(previously studied course DST203 Introduction to Computer Architecture),
- Basics of programming languages

Organizational matters (*see the Requirements document*)

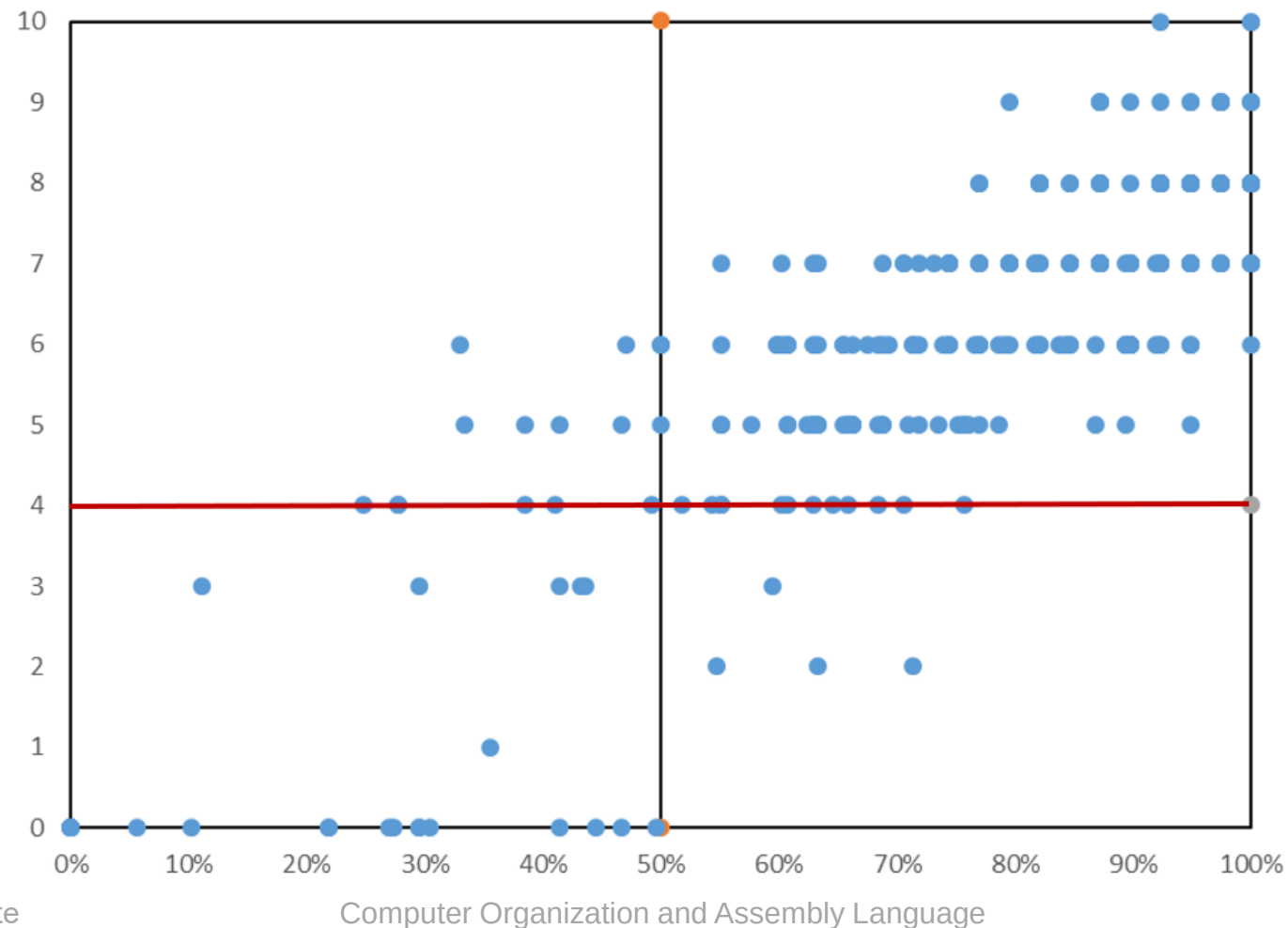
- Attendance of lectures will be monitored
- All lectures will be in English
- Practice classes will be in Latvian for local students and in English for foreign students
- The course page is the main means of information exchange. The course page should be visited at least once a week.
- In the study course the Microsoft Macro Assembler (MASM) language version for the 32-bit environment for x86 processors is used, as well as the 16-bit version in real address mode in the emulation environment. Install the development tools on your home computer. If the student does not have a personal computer that can be used to install and use these tools, then the computers in the RTU computer labs should be used. The use of other software development tools for this study course is not supported.
- If a student uses the support of artificial intelligence tools to complete any of the study tasks, then this must be indicated in the submitted study work, indicating detailed information about the tool used and the part of the solution created by it, which is included in the student's submitted work. If a possible use of artificial intelligence tools without such a reference is detected, then it is qualified as a case of plagiarism and processed in accordance with the procedure established by RTU.

Assessment

- There will be **short tests and mid-semester tests** on dates scheduled in e-course page. It is not possible to rewrite the test except in cases where the student has an objective and documented excuse for absence. At the end of the semester test grade is calculated as weighted average of all test marks. If a student has not participated in the test the test mark is 0 (zero)
- At the end of the course all **individual practice assignments (labs)** are evaluated with one common grade (see Requirements document on the course page for details)
- If less than 4 individual assignments (labs) have been completed, then the average grade for all practice assignments is 0 (zero)
- Average grades of tests and of assignments are calculated before the first exam and cannot be changed later.
- The final examination is evaluated with mark
- The final grade of the study course is calculated according to formula:
final grade = 0.25 x tests + 0.45 x assignments + 0.30 x exam
In the case the student receives non-passing exam mark (i.e. less than 4) the exam mark is used as the final grade (the formula is not applied).
- If the calculated final grade is less than 4, the exam must be re-taken in accordance with RTU regulation.

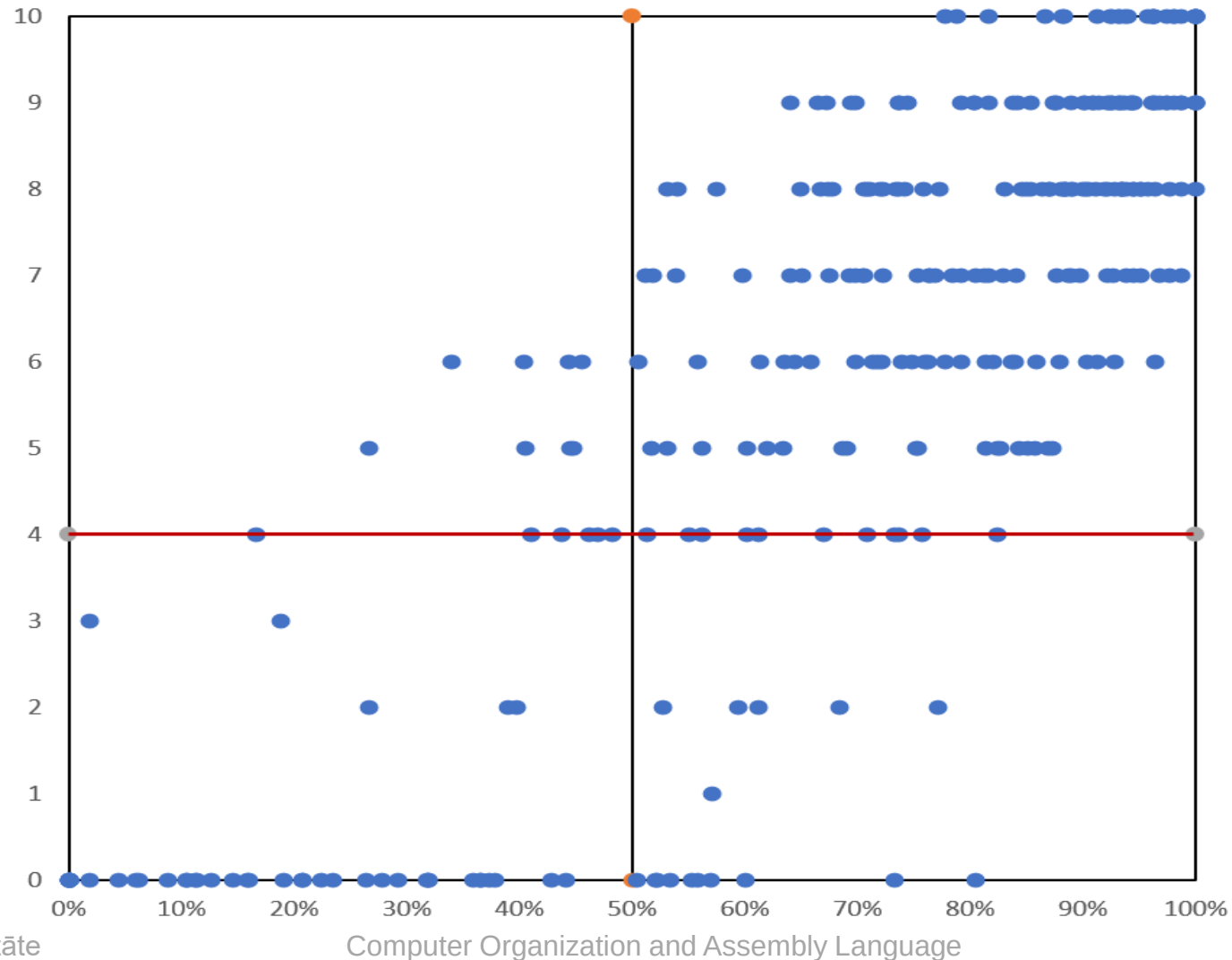
Why is it worth doing your studies regularly?

- Study performance and final grades (course statistics from previous years)



Why is it worth doing your studies regularly?

- Study performance and final grades (course statistics from previous years)



Recommended sources

Slides and code examples – course page

Books

- **Randall Hyde. The Art of Assembly Language (2nd Edition). - No Starch Press, 2010 (see course page, referenced as AoA in course materials)**
- Detmer, Richard C. “Introduction to 80x86 assembly language and computer architecture”, Jones & Bartlett Learning, 2nd edition, 2010
- Van Hoey, Jo. “Beginning X64 assembly programming”, Apress, 2019

Additional

- Kip R. Irvine. Assembly Language for Intel-Based Computers – Prentice Hall, 2006.
- Peter Abel. IBM PC Assembly Language and Programming – Prentice Hall, 2001
- Peter Norton. Peter Norton's Computing Fundamentals – Career Education, 2004

Help files

- Microsoft help

<https://learn.microsoft.com/en-us/cpp/assembler/masm/microsoft-macro-assembler-reference?view=msvc-170>

<https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/x86-instructions>

Tutorials

- Intro to x86 Assembly Language by Davy Wybiral <https://youtu.be/wLXIWKUWpSs>
- Assembly Language Programming Tutorial by Rasim Muratovic <https://www.youtube.com/playlist?list=PLPedo-T7QiNslji329HyTzbKBuCAHwNFC>
- and many more tutorials available on Internet...

Good luck!

